

Authentication & Authorization

Xác Thực và Ủy Quyền

Nhóm 10



Mục Lục

01

Tổng Quan về JWT và OAuth 2.0

02

So Sánh Chi Tiết

03

Các Khái Niệm Cốt Lõi

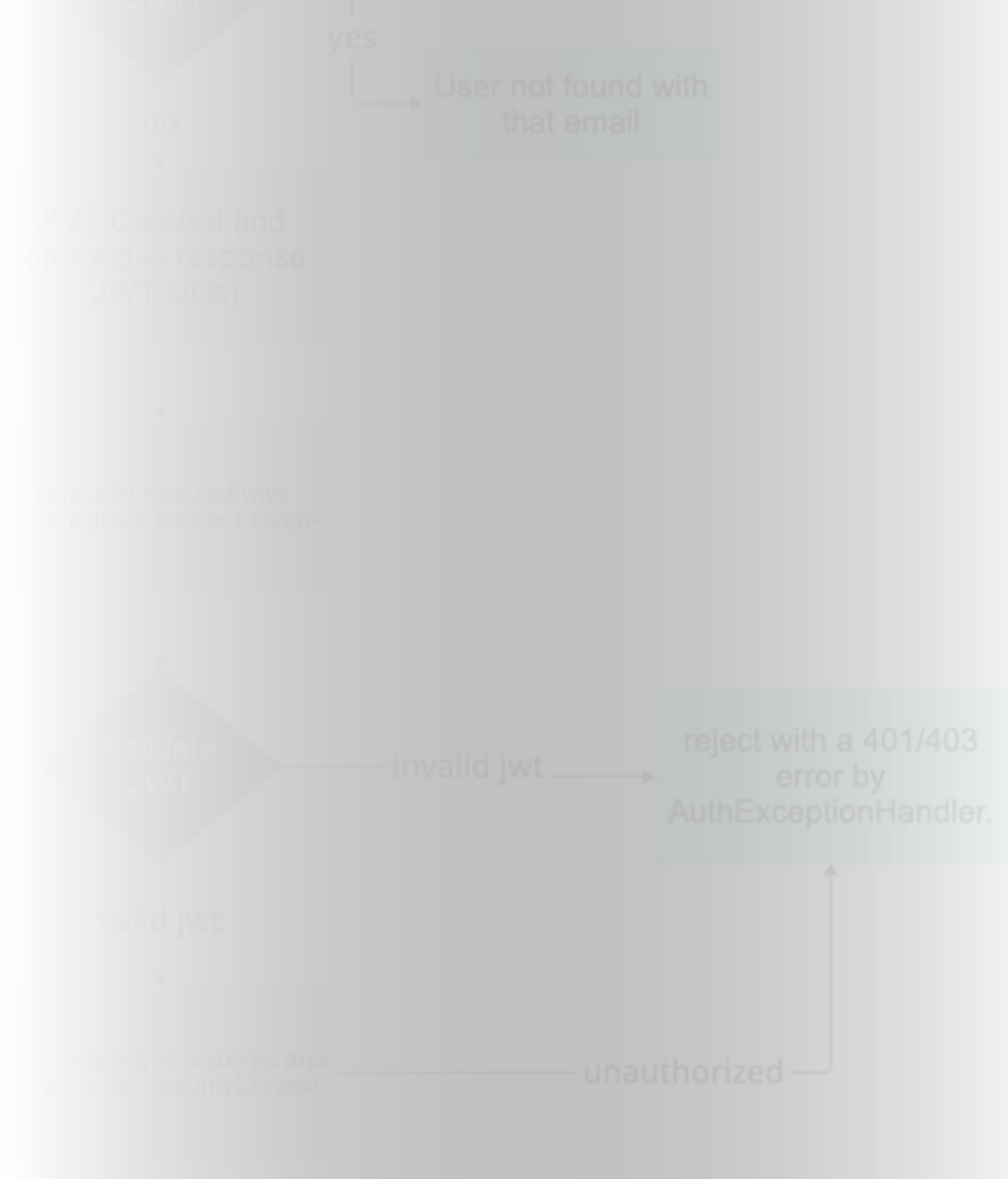
04

Triển Khai JWT Authentication

05

Rủi Ro Bảo Mật

Tổng Quan về JWT và OAuth 2.0



JWT là gì?

JWT (JSON Web Token) là một định dạng token nhỏ gọn, để truyền **claims** (exp, role, ...) một cách an toàn giữa các bên. Đây là chuẩn được sử dụng rộng rãi cho xác thực và ủy quyền.

📁 Cấu Trúc 3 Phần

1 Header

Chứa loại token (JWT) và thuật toán ký (HS256, RS256)

2 Payload

Chứa claims: iss (issuer), sub (subject), exp (expiration), roles, permissions

3 Signature

Chữ ký xác minh tính toàn vẹn: HMAC-SHA256(header + payload, secret)

★ Đặc Điểm Nổi Bật

✓ Stateless

Không cần lưu trữ session trên server

✓ Self-contained

Chứa đầy đủ thông tin để xác thực

✓ Dễ xác thực

Verify signature mà không cần DB lookup

Ví dụ JWT:

```
eyJhbGciOiJIUzI1NiIs  
eyJzdWUiOiIxMjM0NTY3ODkw  
Sf1KxwRJSMeKKF2QT4fwpMe.
```

header.payload.signature

OAuth 2.0 là gì?

OAuth 2.0 là một framework ủy quyền (authorization framework) cho phép ứng dụng bên thứ ba truy cập tài nguyên thay mặt người dùng mà không cần chia sẻ mật khẩu.

Các Thành Phần Chính

Resource Owner

Người sở hữu tài nguyên (end-user)

Client

Ứng dụng yêu cầu truy cập

Authorization Server

Cấp phát và xác thực token

Resource Server

Lưu trữ và bảo vệ tài nguyên

OAuth Flow Cơ Bản

- 1 Client yêu cầu ủy quyền
- 2 User đồng ý và xác thực
- 3 Authorization Server cấp token
- 4 Client dùng token truy cập resource

Mối Quan Hệ với JWT

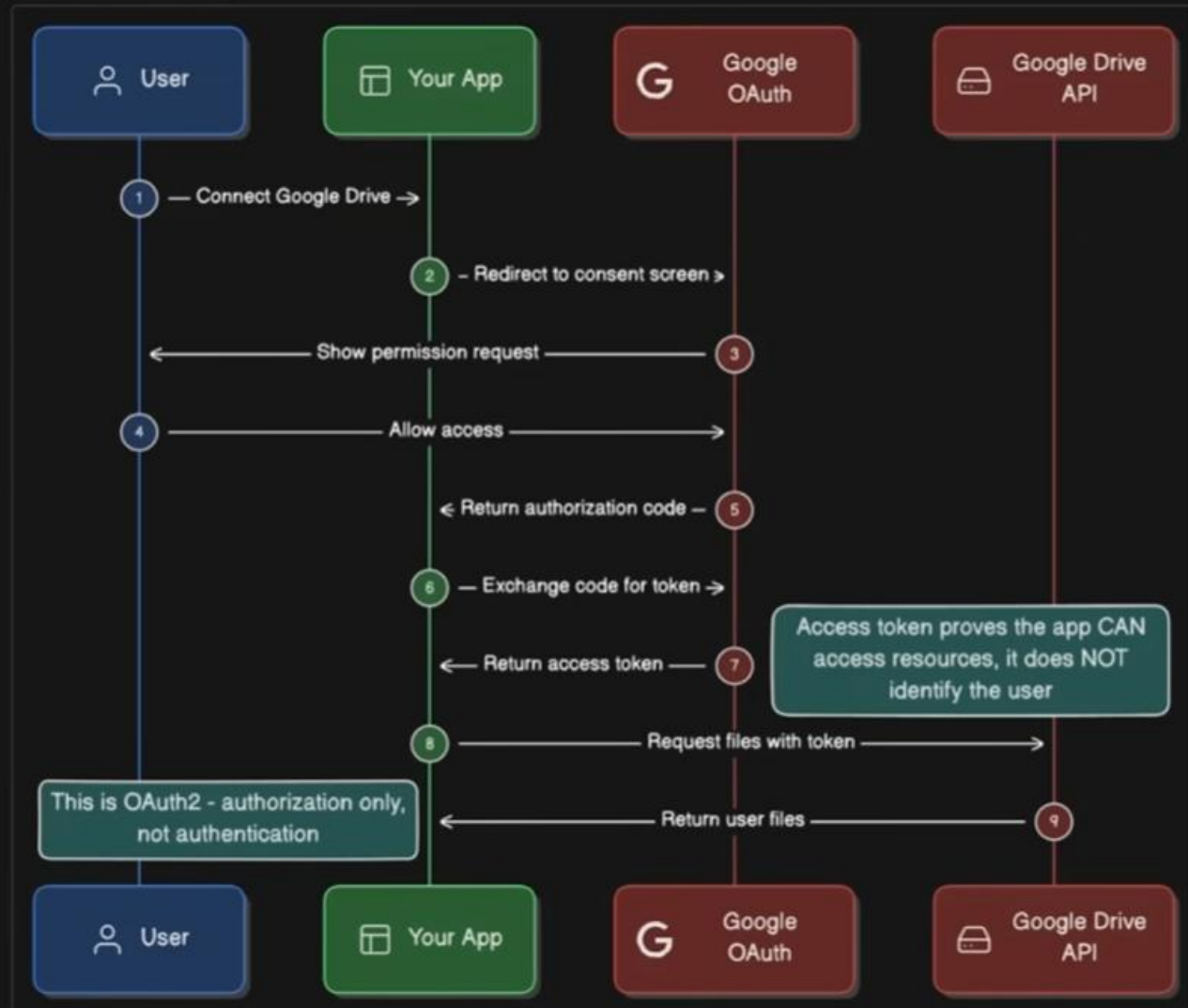
OAuth 2.0 **có thể sử dụng JWT** làm access token hoặc ID token (trong OpenID Connect). Tuy nhiên, OAuth không yêu cầu JWT - opaque tokens cũng là lựa chọn phổ biến.

Use Cases:

- ✓ Social Login (Google, Facebook)
- ✓ Third-party API access
- ✓ Multi-service authorization

OAuth 2.0 Flow

↔ Ví dụ OAuth Flow



So Sánh JWT vs OAuth 2.0

Tiêu Chí	JWT	OAuth 2.0
Bản chất	Định dạng token	Framework ủy quyền
Mục đích chính	Truyền và xác minh claims	Cấp phát và quản lý quyền truy cập
Phạm vi	Cấu trúc và ký token	Quy trình lấy và sử dụng token
State Management	Thường stateless	Có thể stateless hoặc stateful
Định dạng token	Luôn là JWT	JWT hoặc opaque token

✔ Khi nào dùng JWT?

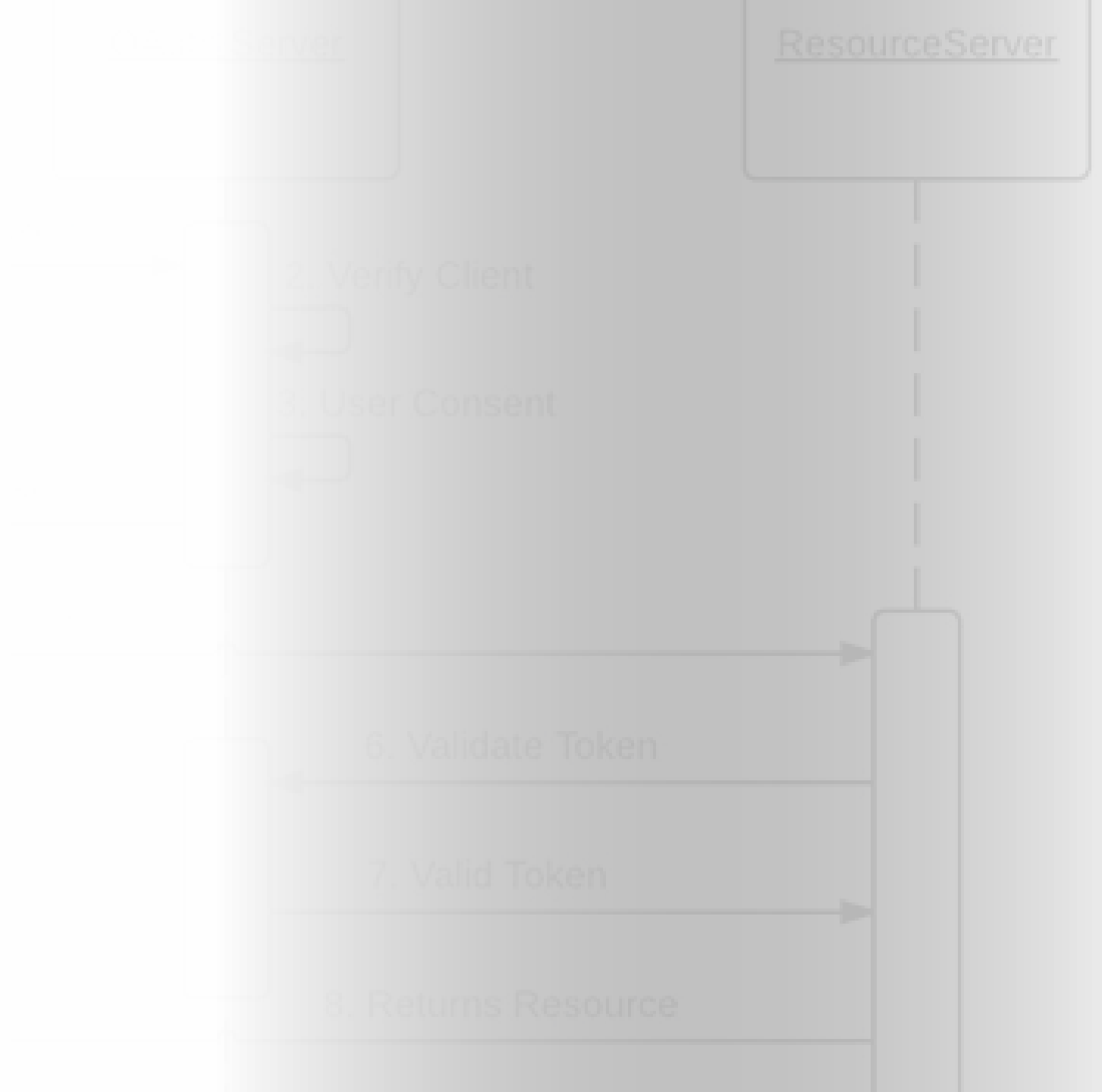
- Ứng dụng đơn giản, cần stateless authentication
- Microservices với nhiều service gọi nhau
- Cần giảm DB lookup cho mỗi request

✔ Khi nào dùng OAuth?

- Third-party access, social login
- Kiểm soát truy cập chi tiết
- Multi-service authorization

Các Khái Niệm Cốt Lõi

Bearer Token, Refresh Token, Scopes và Roles



Bearer Token (Access Token) và Refresh Token



Bearer Token

Token mà **ai sở hữu thì có thể sử dụng** để truy cập tài nguyên.

Đặc điểm

- Short-lived: 5-15 phút
- Chứa trong Authorization: Bearer header
- Self-contained, stateless

Rủi ro

Nếu bị đánh cắp, attacker có thể sử dụng ngay lập tức cho đến khi hết hạn



Refresh Token

Dùng để **lấy access token mới** mà không cần đăng nhập lại. Cho phép session dài hạn với access token ngắn hạn.

Đặc điểm

- Long-lived: 7-30 ngày
- Rất quan trọng
- Chỉ dùng với Authorization Server

Best Practices

- Lưu trong HttpOnly cookies

Scopes và Roles



OAuth Scopes

Chuỗi quyền hạn xác định **ứng dụng có thể làm gì** thay mặt người dùng. Khi user click "Allow" trên consent screen, họ đang grant scopes.

Ví dụ Scopes:

gmail.readonly

drive.file

calendar.readwrite

offline_access

 **Nguyên tắc Least Privilege:** Chỉ request scope thực sự cần thiết



User Roles

Cấp độ truy cập của **người dùng trong ứng dụng**. Được gán bởi administrator.

Ví dụ Roles:

admin

manager

user

guest

Lưu ý: Roles thường được lưu trong JWT payload để authorization.

≠ Scopes vs Roles: Sự Khác Biệt và Kết Hợp

Phạm vi kiểm soát

Scopes: Ứng dụng bên thứ ba

Roles: Người dùng trong hệ thống

Ai cấp phát

Scopes: End-user qua consent

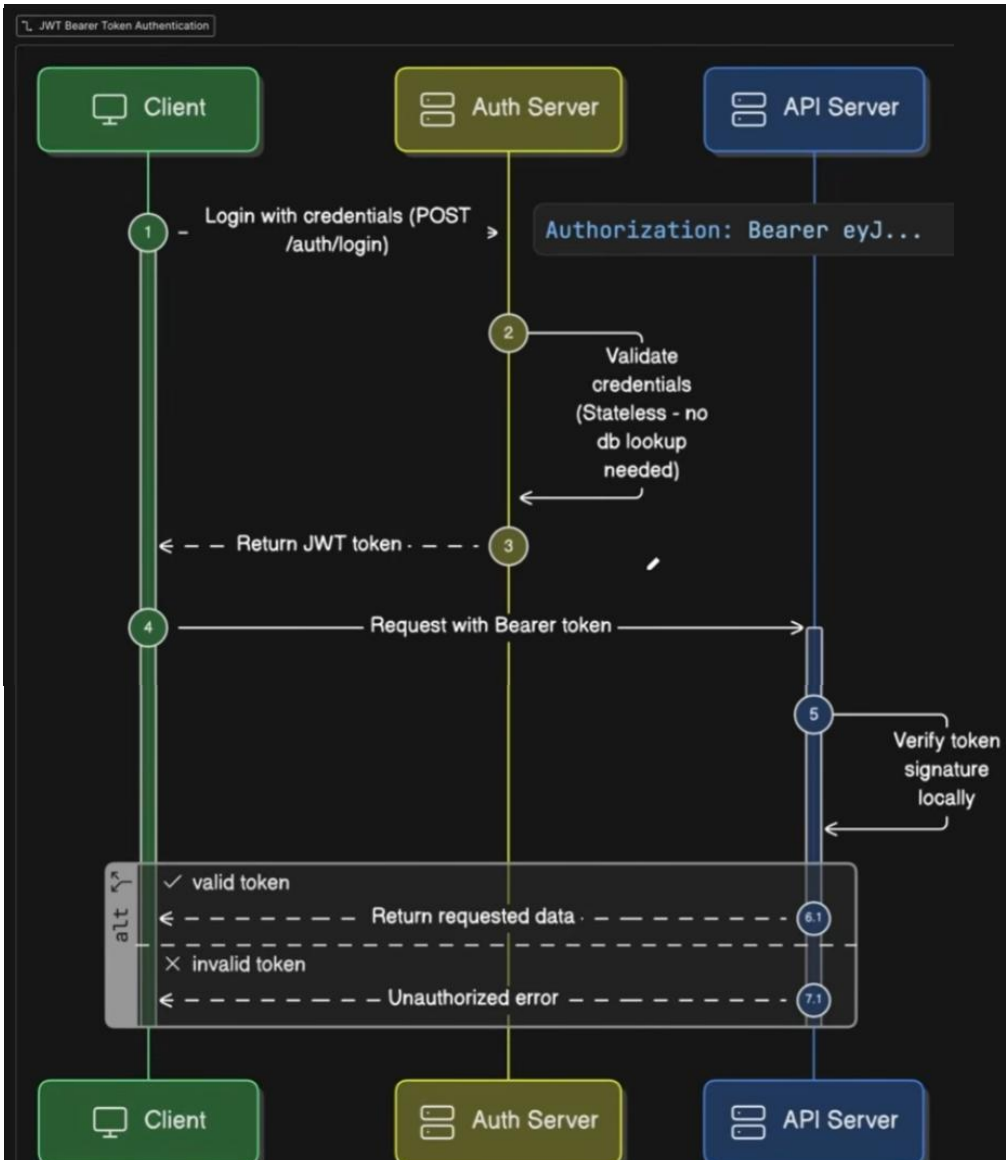
Roles: Administrator

Kết hợp sử dụng

OAuth scopes + user roles trong JWT claims = authorization một cách chi tiết

Triển Khai JWT Authentication

Cài Đặt JWT Authentication Cơ Bản



</> Ví dụ

```

@ns_auth.route('/login')
class Login(Resource):
    @ns_auth.expect(login_model)
    @ns_auth.doc(security=[])
    def post(self):
        """Login to get JWT Token"""
        data = request.json
        username = data.get("username")
        password = data.get("password")

        obj = next((u for u in auths_db if u['username'] == username), None)

        if not check_password_hash(obj["password"], password):
            return wrap_with_metadata_error("invalid-credentials"), 401

        access_token = create_access_token(
            identity=username,
            additional_claims={"role": obj["role"]},
            expires_delta=timedelta(minutes=15)
        )
        refresh_token = create_refresh_token(
            identity=username,
            expires_delta=timedelta(days=30)
        )
        return wrap_with_metadata({
            "access-token": access_token,
            "refresh-token": refresh_token
        }), 200
  
```

Token Lifecycle và Refresh Flow

Access Token Lifecycle

- 1 Issued**
Server tạo và ký token
- 2 Stored**
Lưu trong HttpOnly cookie
- 3 Attached**
Gửi trong Authorization header
- 4 Validated**
Server xác thực signature & claims
- 5 Expired**
exp claim < current time

Refresh Token Flow

1. Token Expired (401)

Access token hết hạn, API trả về 401 Unauthorized

2. Gửi Refresh Request

Client gửi refresh token đến /refresh endpoint

3. Kiểm tra và tạo Token mới

Server kiểm tra refresh token → Cấp access token mới

```
@ns_auth.route('/refresh')
class Refresh(Resource):
    @jwt_required(refresh=True)
    def post(self):
        """Refresh token"""
        identity = get_jwt_identity()
        user = next((u for u in auths_db if u["username"] == identity), None)
        if not user:
            return wrap_with_metadata_error("refresh-token-not-valid"), 400
        access_token = create_access_token(
            identity=identity,
            additional_claims={"role": user['role']}
        )
        return wrap_with_metadata({
            "access_token": access_token
        }), 201
```

Rủi Ro Bảo Mật và Cách Khắc Phục



Token Leakage: Nguyên Nhân và Phòng Chống

⚠ Nguyên Nhân Token Leakage

🦟 XSS (Cross-Site Scripting)

Token trong localStorage/sessionStorage bị JavaScript độc hại đánh cắp

🕸 MITM (Man-in-the-Middle)

Token truyền qua HTTP không mã hóa bị chặn

📄 Logs

Token bị log trong request headers hoặc error logs

🔗 URL Parameters

Token trong query string bị lưu trong browser history

🛡 Phòng Chống Token Leakage

HttpOnly Cookies

Không thể truy cập bởi JavaScript → Chống XSS

```
Set-Cookie: token=xxx; HttpOnly; Secure; SameSite=Strict
```

HTTPS Only

Chỉ truyền token qua HTTPS

Short-lived Tokens

Access token 5-15 phút

Replay Attack: Phát Hiện và Ngăn Chặn

🔄 Replay Attack là gì?

Attacker **chặn và tái sử dụng** valid token để truy cập trái phép. Đặc biệt nguy hiểm.

- Token bị chặn từ network traffic
- Attacker gửi lại token để authenticate
- Server không phân biệt legitimate vs replay

🔍 Phát Hiện Replay Attack

Impossible Travel

Cùng token từ 2 địa điểm cách xa trong thời gian ngắn

Device Fingerprint

Mismatch giữa device ban đầu và device hiện tại

Velocity Anomalies

Request volume bất thường từ một token

Refresh Token Reuse

Phát hiện sử dụng refresh token đã bị invalidate

🔒 Ngăn Chặn Replay Attack

✓ Short-lived Access Tokens

Giảm thời gian token có thể bị replay

✓ Nonce hoặc gắn Timestamp

One-time token cho actions quan trọng

Các Lỗi Hổng JWT Khác



Algorithm Confusion Attack

Attacker đổi từ RS256 sang HS256 hoặc thuật toán khác để forge token.

Khắc phục:

- Whitelist allowed algorithms
- Reject 'none' algorithm (hầu hết các thư viện hiện tại mặc định)



Missing Claim Validation

Không kiểm tra `exp`, `iss`, `aud` → Chấp nhận token hết hạn hoặc sai audience.

Khắc phục:

- Validate tất cả claims
- Kiểm tra exp trước khi xử lý
- Verify iss và aud phù hợp



Weak Signing Keys

Secret yếu (ngắn, đơn giản) bị **brute force** để tạo valid tokens.

Khắc phục:

- Strong random secrets ≥ 256 bits
- Rotate keys regularly
- Không hardcode secrets



Long Token Lifetimes

Token sống quá lâu tăng khả năng bị tấn công nếu bị đánh cắp.

Khắc phục:

- Access token: 5-15 phút
- Refresh token: 7-30 ngày
- Sensitive APIs: 5 phút